

Java Programming

Mai Ngọc Khương
Hà nội, tháng 8 năm 2026

Lớp String & Các Phương Thức Phổ Biến

- Mục tiêu:
 - Biết cách khai báo, khởi tạo và sử dụng các phương thức thông dụng của String. (kiểu dữ liệu được dùng nhiều nhất)

String (chuỗi)

- **Cách khai báo:**

- `String s1 = "Java";`
- `String s2 = new String("Java");`

- **Các phương thức thường dùng:**

- `length()` *Độ dài chuỗi*
- `charAt(int)` *Lấy ký tự tại vị trí*
- `substring(start, end)` *Cắt chuỗi con*
- `indexOf(String)` *Tìm vị trí ký tự/chuỗi*
- `toUpperCase()` *Chuyển thành chữ in hoa*
- `trim()` *Xóa khoảng trắng đầu/cuối*

So sánh chuỗi

- Hiểu sự khác biệt giữa `==` và `.equals()` khi so sánh chuỗi.
- Tránh lỗi phổ biến khi so sánh chuỗi bằng `==`

So sánh địa chỉ (==) và So sánh nội dung (equals)

- **Ví dụ:**

- `String a = "hello";`
- `String b = new String("hello");`

- So sánh địa chỉ (==)

- `System.out.println(a == b); //False`

- So sánh nội dung (equals())

- `System.out.println(a.equals(b)); // True`

→ So sánh chuỗi → luôn dùng `.equals()` để so sánh nội dung/giá trị

Xử lý chuỗi hiệu quả với StringBuilder & StringBuffer

- String là immutable (Không thay đổi)
 - Trong Java, immutable nghĩa là không thể thay đổi sau khi được tạo ra.
- Khi thay đổi chuỗi String, Java không chỉnh sửa nội dung gốc, mà tạo ra một đối tượng chuỗi mới.

Ví dụ:

- `String s = "Hello";`
- `s = s + " World";`
- `System.out.println(s); // Hello World`
- **Giải thích:**
- Ban đầu s tham chiếu đến chuỗi "Hello".
- Khi nối " World" bằng `s + " World"`, Java tạo một đối tượng chuỗi mới chứa "Hello World".
- Biến s không thay đổi nội dung cũ, mà trở sang đối tượng mới.
- Đối tượng "Hello" vẫn tồn tại trong bộ nhớ (chờ GC thu dọn)

Dùng String trong vòng lặp nối chuỗi:

Ví dụ:

```
String result = "";  
for (int i = 0; i < 1000; i++) {  
    result = result + i;  
}
```

- mỗi lần + tạo một đối tượng mới,
- với 1000 vòng lặp Java tạo ra hàng trăm đối tượng String mới
- tốn RAM và gây ra chương trình chậm

StringBuilder

- Cả hai đều là lớp hỗ trợ thao tác với chuỗi có thể thay đổi (mutable), nhằm giải quyết hạn chế của lớp String (immutable).
- Nên dùng khi:
- Bạn cần **nối chuỗi nhiều lần** (trong vòng lặp, xử lý văn bản).
- Muốn tăng hiệu suất và **tiết kiệm bộ nhớ** hơn so với dùng String.
- `StringBuilder sb = new StringBuilder("Hello");`

Các phương thức thường dùng:

Phương thức	Mô tả	Ví dụ
append(String)	Nối thêm vào cuối chuỗi	sb.append("abc")
insert(int, String)	Chèn chuỗi tại vị trí chỉ định	sb.insert(0, ">>> ")
delete(start, end)	Xóa chuỗi con	sb.delete(2, 4)
replace(start, end, str)	Thay thế chuỗi con	sb.replace(0, 4, "Hi")
reverse()	Đảo ngược chuỗi	sb.reverse()

// Dùng String (chậm)

```
String s = "";
for (int i = 0; i < 10000; i++) {
    s = s + i;
}
```

// Dùng StringBuilder (nhanh)

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10000; i++) {
    sb.append(i);
}
```